SAD Laboratory Activity # 3

Data Flow Diagrams

MEMBERS:

-Cuario, John Lloyd M.
-Loterte, Eljohn Paulo C.
-Napay, Victor Noel A.
-Rebancos, Sean Jerve Ll.
-Relos, Kiel Hedrix V.

Part A: Analysis of the Existing System

The current business process of the Fe Galang Da Silva Apartment was reviewed and broken down into the standard DFD components below, This is based on our capstone study “Hivelet: A Web-Based Apartment Management System for Fe Galang Da Silva Boarding House”.

External Entities

- Tenant — current boarding house resident who logs in, pays rent, and reports maintenance issues.
- Public User / Prospective Renter — a non-tenant visitor who submits room booking inquiries.
- Administrator — the landlady managing tenants, bookings, finances, and tickets.
- Adyen Payment Gateway — the optional external service used for online payment recording.

Major Processes

- 1.0 Manage Tenant and Room Records
- 2.0 Process Booking and Reservation
- 3.0 Track Financial Transactions
- 4.0 Manage Maintenance Tickets
- 5.0 Authenticate and Authorize Users (RBAC)

Data Stores

- D1 Tenant Records
- D2 Room/Unit Records
- D3 Booking/Reservation Records
- D4 Financial Transactions
- D5 Maintenance Tickets
- D6 User Accounts (login credentials and roles)

Key Inputs

- Tenant information
- Login credentials
- Booking inquiry details
- Payment records or proof of payment
- Maintenance issue reports
- Room/unit data
- Ticket status updates.

Key Outputs

- Booking confirmation or rejection
- Financial reports and cash flow summaries
- Maintenance ticket notifications
- Tenant account information
- Room availability status
- Payment confirmation

Key Data Flows

- Tenant data

- Booking request
- Booking response
- Payment data
- Payment confirmation
- Transaction record
- Maintenance request
- Ticket update
- Login credentials
- Access token
- Room availability query
- Financial report
- Notifications

Part B: Context Diagram

The Context Diagram represents Hivelet as a single process, showing only the system boundary and the four external entities it exchanges data with.

Tenants send login, booking, payment, and maintenance data into the system and receive confirmations and account information back. Public users only interact through booking inquiries, since they do not yet have system accounts. Administrators exchange approvals, room and tenant data, and receive reports and notifications. The optional Adyen gateway exchanges payment requests and status with the financial tracking process inside the system boundary.

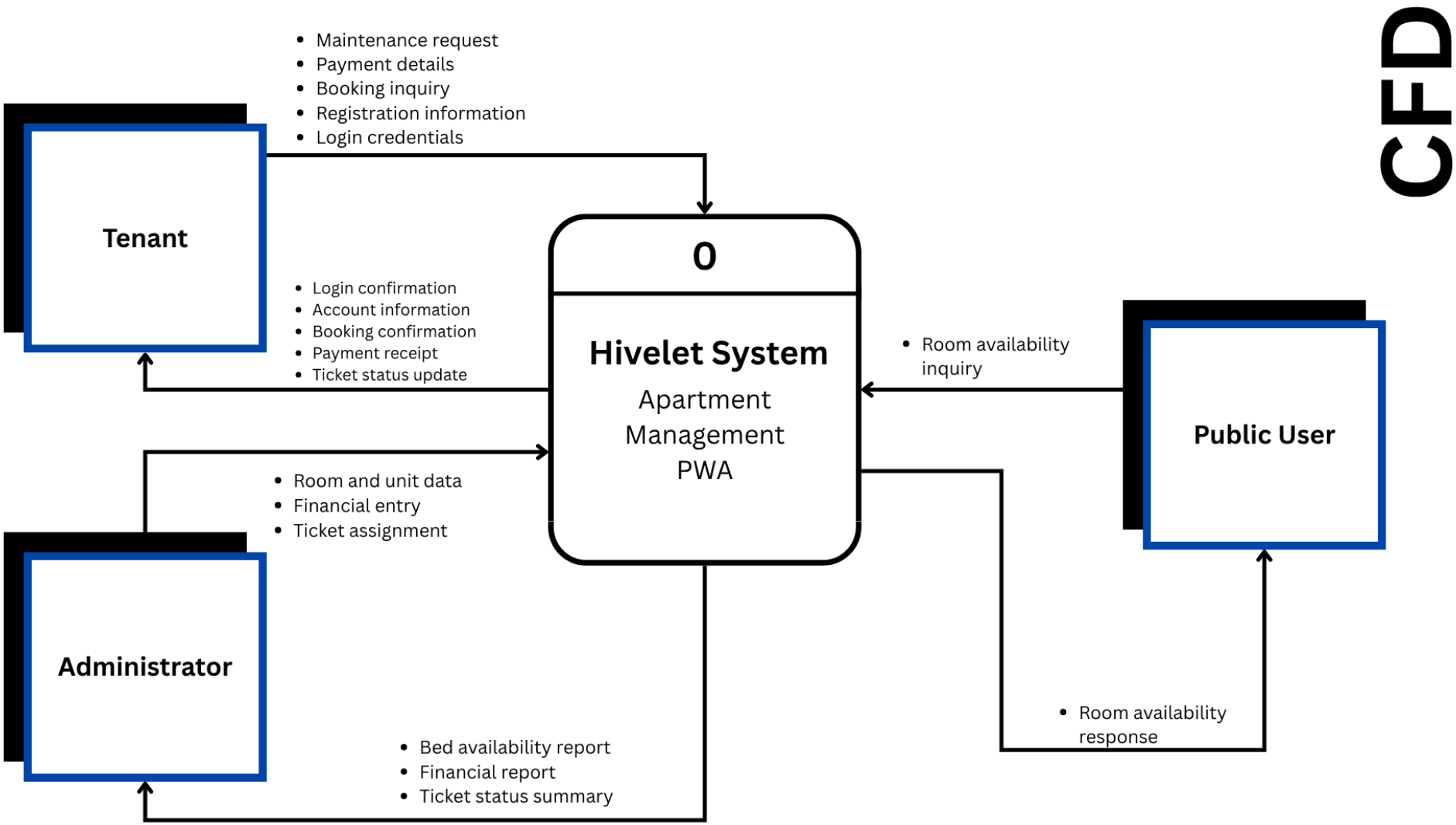


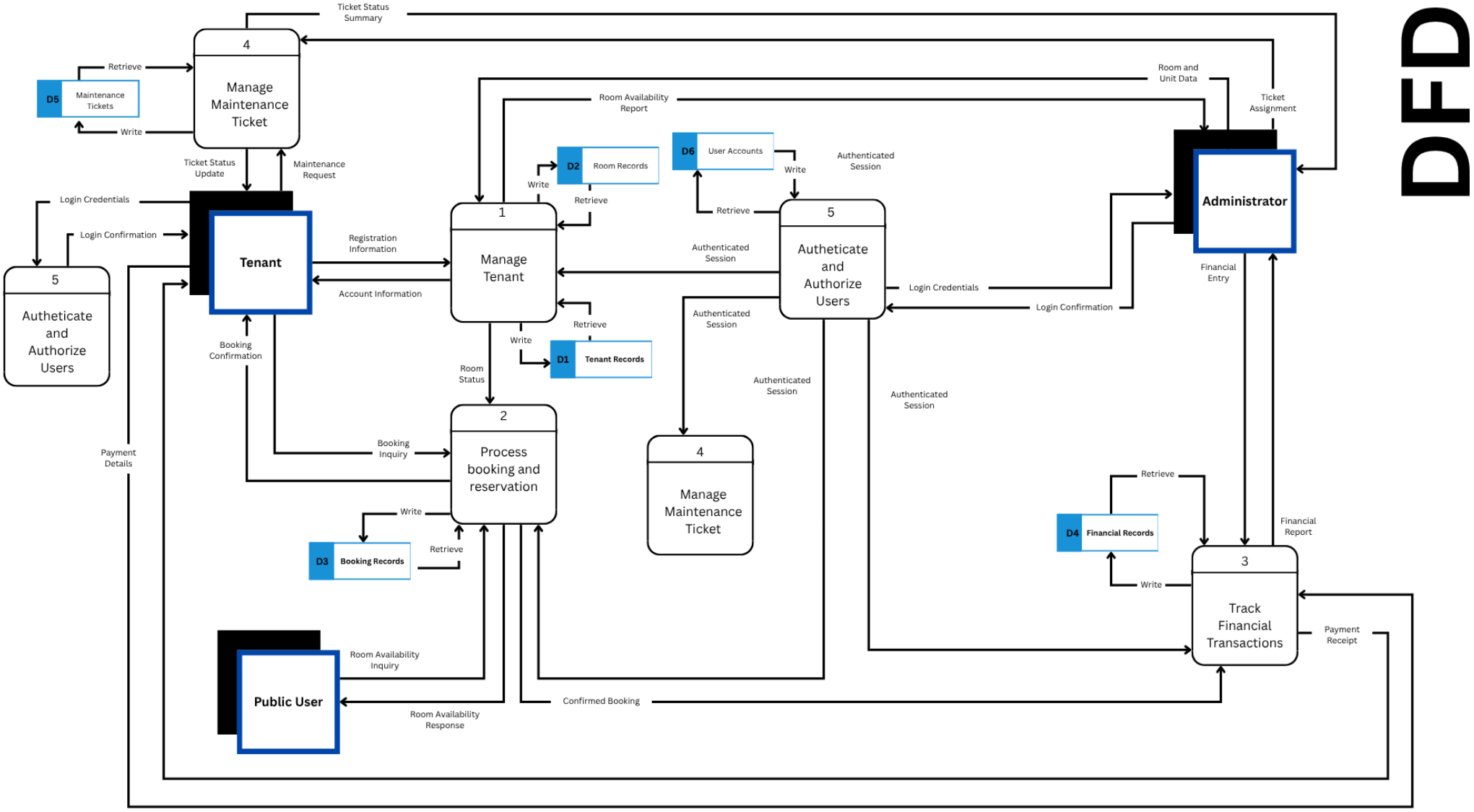
Figure 1. Context Diagram of the Hivelet system

Part C: Logical Diagram 0 (Level 0 DFD)

Diagram 0 decomposes the single Process 0 from the Context Diagram into Hivelet's five major processes, numbered sequentially from 1.0 to 5.0, and introduces the six data stores that support them.

Figure 2 Process 1.0 receives tenant information and updates D1 and D2, then passes room status to 2.0 so booking decisions reflect real-time availability. Process 2.0 takes public booking inquiries, writes to D3, and forwards confirmed bookings to 3.0 for billing setup. Process 3.0 records all transactions in D4 and, when the optional online payment feature is used, exchanges payment data with the Adyen gateway. Process 4.0 manages tenant-submitted tickets in D5, while Process 5.0 authenticates every login attempt against D6 before any tenant or administrator can reach the other modulels

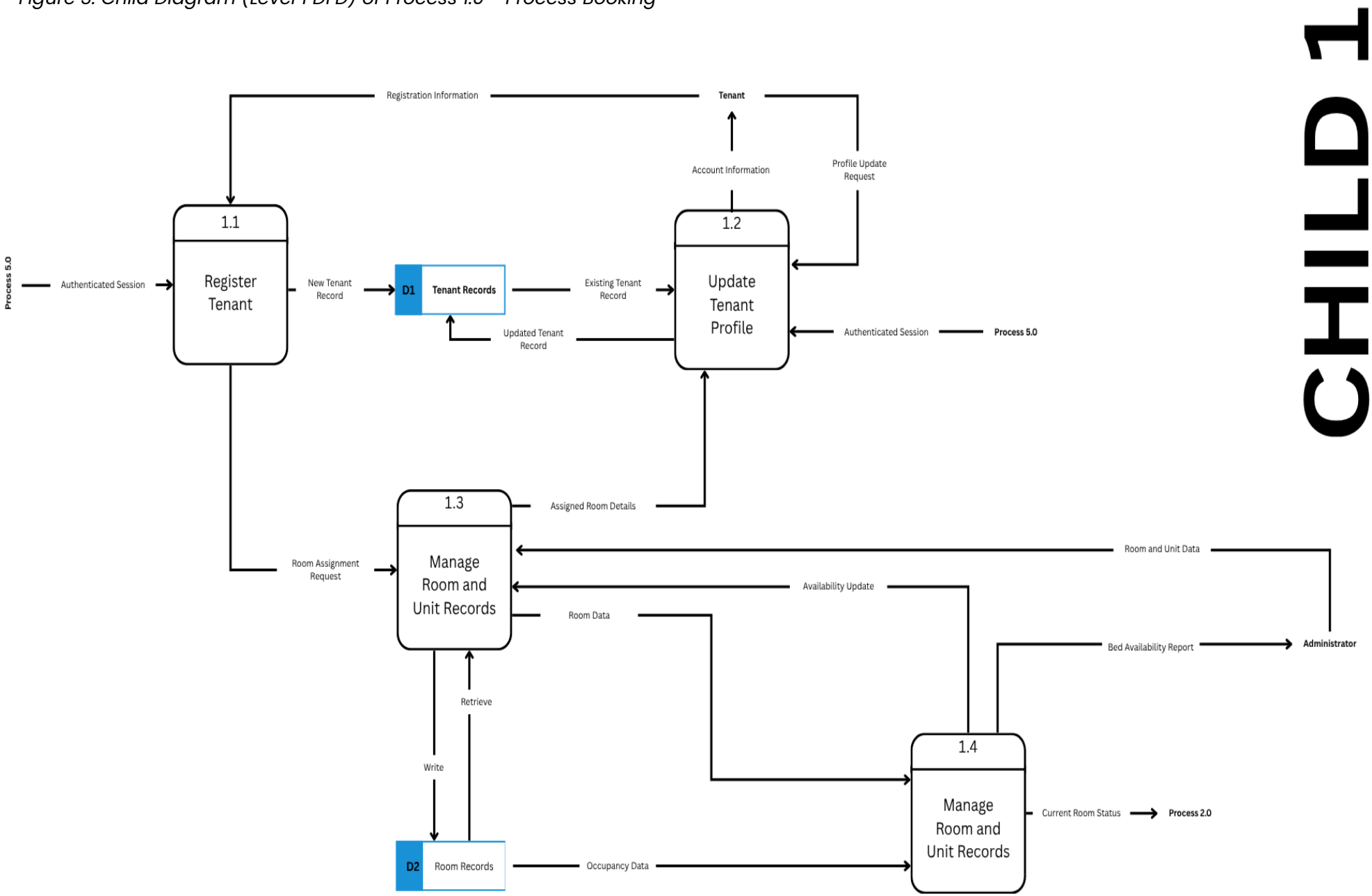
Figure 2. Diagram 0 (Level 0 DFD) of the Hivelet system



Part D: Child Diagram (Level 1 DFD)

Process 1.0 (Manage Tenant and Room Records) was selected for decomposition because it is the starting point for data flows. 1.1 Register and Onboard Tenant creates the initial record in D1. 1.2 Update Tenant Profile and Status edits D1 and confirms changes back to the tenant.1.3 Manage Room and Unit Records lets the administrator add/edit units in D2. 1.4 Monitor Bed Availability queries D2 and reports occupancy to the administrator.

Figure 3. Child Diagram (Level 1 DFD) of Process 1.0 - Process Booking

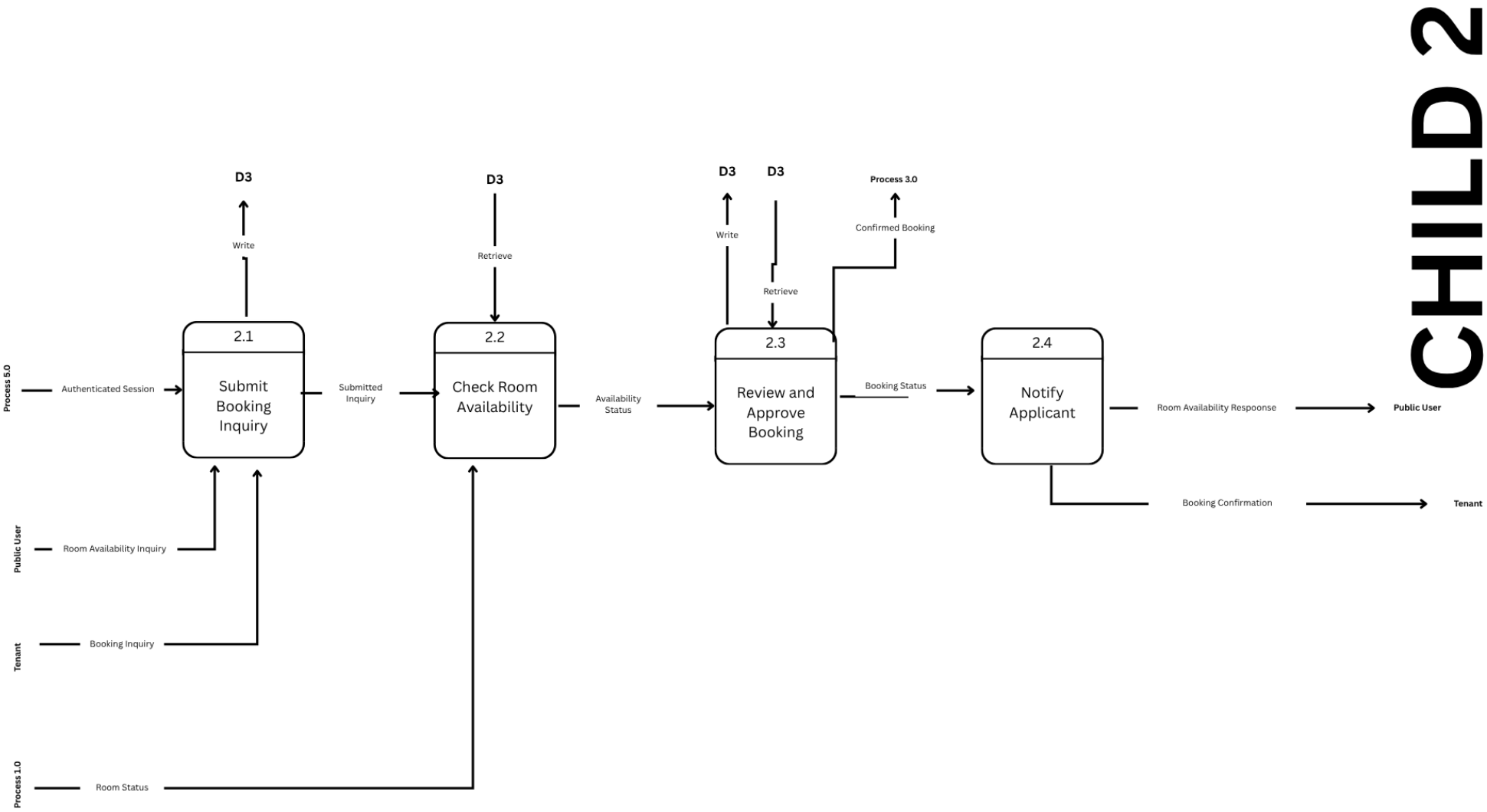


Child Diagram (Level 1 DFD)

Another process is chosen, Process 2.0 (Process Booking and Reservation) was selected for decomposition because it is one of the operationally distinct module that spans an unauthenticated public-facing entity (the prospective renter) through to an administrator decision, making it a useful illustration of how a parent process expands into balanced subprocesses 2.1 through 2.4.

2.1 Submit Booking Inquiry captures the public user's request and writes it to D3. 2.2 Check Room Availability cross-references D2 and D3 to determine whether a unit is open for the requested period. 2.3 Review and Approve Booking is where the administrator makes the accept/reject decision, updating D3 with the final status. 2.4 Notify Applicant then reads that status and sends the booking confirmation back out to the public user. The inputs and outputs at the edges of this diagram, the inquiry coming in from the public user and the approval decision coming in from the administrator, match exactly what entered and exited Process 2.0 in Diagram 0, keeping the decomposition balanced.

Figure 4. Child Diagram (Level 2 DFD) of Process 2.0 - Process Booking

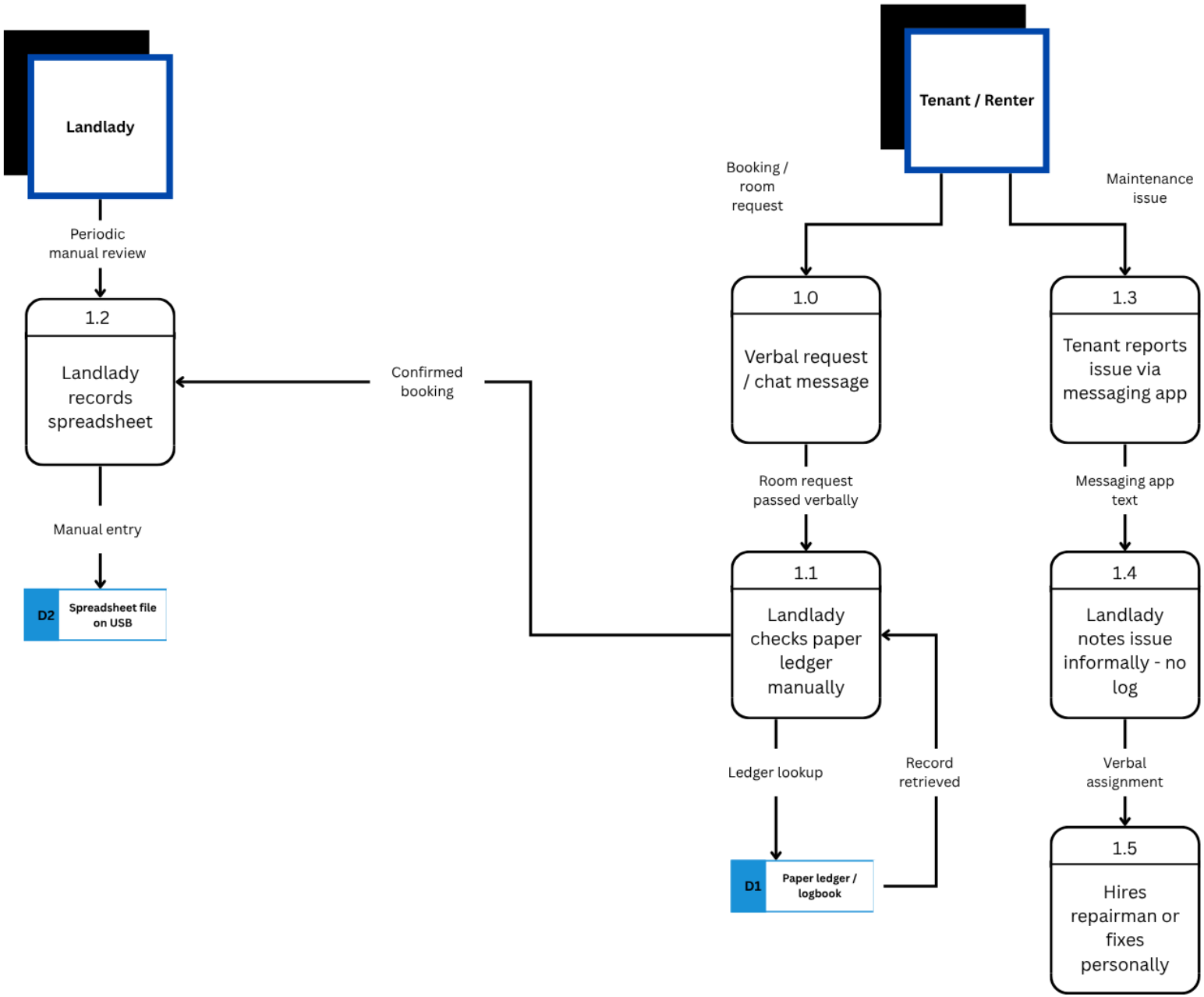


Part E: Physical (Current) DFD

This diagram illustrates how the apartment is actually managed today, before Hivelet, based on the operational gaps described in the capstone's background of the study: fragmented manual record-keeping, removable storage devices, and informal communication channels.

On the booking side, a renter's request is communicated verbally or by chat message, after which the landlady manually checks a paper ledger and records the confirmed booking into a spreadsheet kept on a removable USB drive, a process exposed to accidental deletion, file corruption, and lack of version control. On the maintenance side, tenants report issues through a messaging application; the landlady notes the issue informally, with no centralized log, and either personally repairs the unit or verbally assigns it to a hired repairman. Because every step depends on manual transcription and informal channels, there is no structured tracking, audit trail, or guaranteed follow-up precisely the fragmentation Hivelet's logical design is built to replace.

Figure 5. Physical DFD of the current (manual) system



PHYSICAL